

ASSESSMENT TOOL - 01.

1) LIFO Data Structure - Stack.

Given,

Array size = 5

Operations:-

Push(45)

Push(22)

Push(11)

Push(77)

Pop()

Push(44)

Push(33)

Pop()

Pop()

Pop()

PUSH(45)

45 $\leftarrow$ TOP

PUSH(22)

22 $\leftarrow$ TOP

45

PUSH(11)

11 $\leftarrow$ TOP

22

45

PUSH(77)

77 $\leftarrow$ TOP

11

22

45

POP()

77 is removed.

11 $\leftarrow$ TOP

22

45

PUSH(44)

44 $\leftarrow$ TOP

11

22

45

PUSH(33)

33 $\leftarrow$ TOP

44

11

22

45

POP()

33 is removed

POP()

44 is removed.

FINAL STACK:-

22 $\leftarrow$ TOP

45

Peak/Top element = 22

Index position = 04.

2) Circular Queue Operations.

Queue size = 5

OPERATIONS:-

Enqueue (20), Enqueue (10), Enqueue (30),

Dequeue (), Enqueue (80), Dequeue (),

Enqueue (90), Enqueue (100), Dequeue (),

Dequeue (), Enqueue (60), Enqueue (90)

Circular queue follows FIFO (First In First Out)

Initially:-

Front = -1, Rear = -1

Operation	Queue contents	Front	Rear
Enqueue (20)	20	0	0
Enqueue (10)	20, 10	0	1
Enqueue (30)	20, 10, 30	0	2
Dequeue ()	10, 30	1	2
Enqueue (80)	10, 30, 80	1	3
Dequeue ()	30, 80	2	3
Enqueue (90)	30, 80, 90	2	4
Enqueue (100)	30, 80, 90, 100	2	0
Dequeue ()	80, 90, 100	3	0
Dequeue ()	90, 100	4	0

Enqueue(60)	90, 100, 60	4	1
Enqueue(90)	90, 100, 60, 90	4	2

FINAL CIRCULAR QUEUE:



Front element = 90 at position 4

Rear element = 90 at position 2

3) Binary Search Algorithm.

Given 10 sorted elements:-

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

Algorithm:-

* Set low = 0

* Set high = n-1.

* Calculate $mid = \frac{low + high}{2}$

* Compare the key with A[mid].

* If key = A[mid], element is found

* If key < A[mid], search the left half.

* If key > A[mid], search the right half

* Repeat until low > high

CASE: 01:- key = 111

Array:-

66 111 212 323 432 543 643 649 777 888

Initially:

$$\text{low} = 0, \text{high} = 9$$

$$\text{mid} = \frac{0+9}{2} = 4$$

$$A[4] = 432$$

Since:

$$111 < 432$$

$$\text{low} = 0, \text{high} = 3$$

$$\text{mid} = \frac{0+3}{2} = 1$$

$$A[1] = 111$$

Therefore, 111 found at index 01.

Position = 2.

CASE: 02:- key = 888

Initially:

$$\text{low} = 0, \text{high} = 9$$

$$\text{mid} = 4$$

$$A[4] = 432$$

Since:- $888 > 432$

$$\text{low} = 5, \text{high} = 9$$

$$\text{mid} = 7$$

$$A[7] = 649$$

$$888 > 649$$

Sol:-

low = 8, high = 9

mid = 8

$A[8] = 777$

$888 > 777$.

Sol:-

low = 9

mid = 9

$A[9] = 888$

Therefore:-

888 found at index 9

Position = 10

CASE:03:- key = 999.

Initially:-

low = 0, high = 9.

Finally:-

low = 10, high = 9

Since:

low > high

999 is not found.

COMPLEXITY:-

Best case :- $O(1)$

Worst case :- $O(\log n)$

Space complexity :- $O(1)$

4) Infix to Postfix and Postfix Evaluation.

A) Infix to Postfix.

Given:-

$$B - (C/D + (E \% F * G) / H) * J$$

Operator precedence:-

1. ()

2. $\%$, $/$, $*$

3. $+$, $-$

Expression:-

$$B - (C/D + (E \% F * G) / H) * J$$

PART:01:-

C/D

postfix:- $CD/$

PART:02:-

$E \% F$

$EF\%$

$EF\%G*$

$EF\%G*H/$

$CD/EF\%G*H/+$

$CD/EF\%G*H/+J*$

POSTFIX:-

$BCD/EF\%G*H/+J* -$

B) Evaluate $9\ 3\ 8\ 4\ /\ * +$

using stack:

TOKEN	OPERATION	STACK
9	Push	9
3	Push	9, 3
8	Push	9, 3, 8
4	Push	9, 3, 8, 4
/	$8 \div 4$	9, 3, 2
*	3×2	9, 6
+	$9 + 6$	15

Therefore 15

c) Evaluate $6\ 2\ +\ 6\ 3\ /\ *$

TOKEN	OPERATION	STACK
6	Push	6
2	Push	6, 2
+	$6 + 2$	8
6	Push	8, 6
3	Push	8, 6, 3
/	$6 \div 3$	8, 2
*	8×2	16

Therefore: 16

5) Queue Operations:-

Queue size = 5

Operation	Queue	Front	Rear
Enqueue (21)	21	0	0
Enqueue (67)	21, 67	0	1
Enqueue (89)	21, 67, 89	0	2
Deque()	67, 89	1	2
Enqueue (15)	67, 89, 15	1	3
Enqueue (30)	67, 89, 15, 30	1	4
Enqueue (52)	Overflow	1	4
Deque()	89, 15, 30	2	4
Deque()	15, 30	3	4
Deque()	30	4	4
Deque()	Empty	-1	-1

Final Queue = Empty.

6) Balanced Parentheses:-

$$(1) \quad [9 * (k + 8 - \{6 + e\} + 77) / 100$$

opening brackets:-

[({

closing brackets:-

})

∴ NOT BALANCED

$$(ii) (((a * b) + [u/2] - d)$$

Opening brackets :-

((([

Closing brackets :

)])

NOT BALANCED

7) Prefix and Postfix Conversion.

Given :-

$$((a+b) + c * (d+e) + f) * (g+h)$$

We need :-

1. Prefix

2. Postfix

$$(a+b)$$

$$+ab$$

$$ab+$$

$$(d+e)$$

$$+de, de+$$

$$c * +de$$

$$cde+*$$

$$((a+b) + c * (d+e) + f)$$

$$((a+b) + (c * (d+e))) + f$$

$$+++ab * cde f$$

$$ab+cde+*+f+$$

$$(g+h)$$

$$+gh$$

$$gh+$$